Week 1: Self-Attention from Scratch

Reading Project: Near-Optimal Real-Time Personalization with Simple Transformers

Goal for the week

By the end of this week you should be able to write down the self-attention operation from memory, implement it in NumPy without an autodiff library, and explain in plain language what each of the three matrices $\boldsymbol{Q}$, $\boldsymbol{K}$, $\boldsymbol{V}$ contributes.

Nothing about the paper yet. This week is purely about the mechanism the paper is built on. Everything you write this week will be imported by code you write in later weeks, so treat it as the first module of a library rather than a throwaway script.

Budget: 6–8 hours. Roughly 2 hours reading, 4 hours coding, 1 hour writing.

1 Reading

1. Andrej Karpathy, *Let's build GPT: from scratch, in code, spelled out.* Watch the segment on self-attention (roughly the 55–85 minute mark). Watch it with an editor open and type the code as he does; do not just watch. https://www.youtube.com/watch?v=kCc8FmEb1nY

2. Vaswani et al., *Attention Is All You Need* (2017), **Section 3.2 only**. Skip the rest of the paper. https://arxiv.org/abs/1706.03762

Optional if you want a second pass in a different style: Jay Alammar, *The Illustrated Transformer*.

2 The object we are building

Fix an input dimension n, an embedding dimension d_{kq}, and an output dimension d_v. Given matrices

$$\boldsymbol{Q}, \boldsymbol{K} \in \mathbb{R}^{n \times d_{kq}}, \qquad \boldsymbol{V} \in \mathbb{R}^{n \times d_v},$$

the *row-wise softmax* of a matrix $A \in \mathbb{R}^{n \times d}$ is

$$\mathrm{softmax}(A)_{i,j} \;=\; \frac{\exp(A_{i,j})}{\sum_{j'=1}^{d} \exp(A_{i,j'})},$$

so that every row of $\mathrm{softmax}(A)$ is non-negative and sums to one.

A *self-attention layer* is the map $\mathrm{SA}_{\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V}} : \mathbb{R}^{n \times n} \to \mathbb{R}^{n \times d_v}$ given by

$$\boxed{\mathrm{SA}_{\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V}}(\boldsymbol{X}) \;=\; \mathrm{softmax}\big((\boldsymbol{X}\boldsymbol{Q})(\boldsymbol{X}\boldsymbol{K})^{\top}\big)\,\boldsymbol{X}\boldsymbol{V}.}$$

Write $\boldsymbol{W} = \mathrm{softmax}\big((\boldsymbol{XQ})(\boldsymbol{XK})^\top\big) \in \mathbb{R}^{n\times n}$. Since each row of $\boldsymbol{W}$ sums to one, the i-th output row is a weighted average of the rows of $\boldsymbol{XV}$:

$$\mathrm{SA}_{\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V}}(\boldsymbol{X})_i \;=\; \sum_{j=1}^{n} \boldsymbol{W}_{ij}\,(\boldsymbol{XV})_j.$$

That is the whole mechanism. Everything else in a transformer is layers of this plus point-wise transformations.

3 Questions

Answer these in your own words. Short is fine; three or four sentences each. Do not copy phrasing from the video or the paper — if you cannot say it in your own words you do not yet understand it.

Q1. Shapes

Verify, by tracking dimensions through the formula, that $\mathrm{SA}_{\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V}}(\boldsymbol{X})$ is an $n \times d_v$ matrix. State the shape of every intermediate product.

X has shape $n \times n$, Q and K have shape $n \times d_{kq}$, and V has shape $n \times d_v$. Then XQ is $(n \times n)(n \times d_{kq}) = n \times d_{kq}$, and XK is also $n \times d_{kq}$, so $(XK)^\top$ is $d_{kq} \times n$. Multiplying gives $(XQ)(XK)^\top$ of shape $(n \times d_{kq})(d_{kq} \times n) = n \times n$. Softmax is applied row-wise so it doesn't change the shape, leaving W as $n \times n$. Meanwhile XV is $(n \times n)(n \times d_v) = n \times d_v$. Finally $W(XV)$ is $(n \times n)(n \times d_v) = n \times d_v$ which matches the claimed output shape.

Q2. Why three matrices?

$\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ play different roles. Explain what each one does. A useful framing: if row i of the output is a weighted average of value vectors, then which matrix determines *what is being averaged*, and which two determine *the weights*?

V determines what is being averaged. Then XV is the set of value vectors that get weighted and summed to produce each output row. Q and K together determine the weights. Row i of Q (query i) is compared against every row of K via a dot product, so Q says what a token is looking for and K says what each token offers. Their interaction sets how much weight token i puts on token j before the values are combined.

Q3. Why softmax and not something else?

Two properties of the softmax matter here: rows are non-negative, and rows sum to one. Name one thing that would break if we dropped the normalization and used raw exp values as weights.

We have to have row-wise normalization because that is what turns the scores into a weighted average in the first place. Without dividing by the row sum, the rows of W would not sum to 1, so each output row would just be an unnormalized sum of value vectors that grows with n (the number of tokens) rather than a proper average. This ultimately breaks scale consistency because some rows with more large positive scores would produce outputs with much larger magnitude only due to having longer sequence lengths and not because of anything meaningful in the data.

Q4. Asymmetry

Is $\boldsymbol{W}$ symmetric in general? That is, does $\boldsymbol{W}_{ij} = \boldsymbol{W}_{ji}$? Justify your answer, and say what it would take for $\boldsymbol{W}$ to be symmetric.

W is not symmetric in general. W_{ij} comes from softmax applied to row i of $(XQ)(XK)^\top$, normalized by the sum over row i, while W_{ji} comes from a completely different row (row j) normalized by a different sum. Even if $Q = K$, making the raw dot products $q_i \cdot k_j = q_j \cdot k_i$ symmetric before softmax, the row-wise normalization uses a different denominator for each row, so $W_{ij} = W_{ji}$ would only hold by coincidence. Therefore it is not symmetric.

Q5. Permutation invariance

Suppose you permute the rows of $\boldsymbol{X}$ by a permutation matrix P, i.e. replace $\boldsymbol{X}$ by $P\boldsymbol{X}$. What happens to $\mathrm{SA}_{\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V}}(\boldsymbol{X})$? Work it out symbolically, then check it numerically in your code. This question is not incidental: it is the reason positional encodings exist, and it will matter when we get to the paper's model.

Permuting the rows of X by P just permutes the rows of the output the same way: $\mathrm{SA}(PX) = P \cdot \mathrm{SA}(X)$. This is because $(PX)Q = P(XQ)$ and $(PX)K = P(XK)$, so $W' = \mathrm{softmax}(P(XQ)(XK)^\top P^\top) = PWP^\top$ (softmax acts row-wise, so row permutations pass through it). Then $\mathrm{SA}(PX) = W'(PX)V = PWP^\top P(XV) = PW(XV) = P \cdot \mathrm{SA}(X)$, since $P^\top P = I$. This ultimately reveals that self-attention has no notion of word order on its own. It can only react to which rows exist but not their positions. Due to this lack of word order information positional encodings have to be added separately later.

4 Code

Create a repository (git, private, shared with me) with this structure:

```
transformer-personalization/
  src/
    attention.py
  tests/
    test_attention.py
  notebook.md
  README.md
```

Task 1: `softmax`

Implement row-wise softmax in NumPy. Signature:

```
def softmax(A: np.ndarray) -> np.ndarray:
    """Row-wise softmax. A has shape (n, d); returns shape (n, d)."""
```

Do this the numerically stable way. Subtract the row maximum before exponentiating. If you do not, try your implementation on a matrix with entries around 1000 and see what happens; then fix it. Record what you observed in `notebook.md` — this is a mistake worth having made once.

Task 2: `self_attention`

Implement the layer. No PyTorch, no autodiff, just NumPy.

```
def self_attention(X: np.ndarray,
                   Q: np.ndarray,
                   K: np.ndarray,
                   V: np.ndarray) -> np.ndarray:
    """Returns softmax((XQ)(XK)^T) XV, of shape (n, d_v)."""
```

Also expose the attention weight matrix, since we will need to inspect it in almost every later week:

```
def attention_weights(X, Q, K) -> np.ndarray:
    """Returns W = softmax((XQ)(XK)^T), of shape (n, n)."""
```

Task 3: tests

In `tests/test_attention.py`, write tests that check:

1. Every row of `attention_weights` sums to 1 (to within 10^{-10}) and every entry is non-negative.
2. Output shape is (n, d_v) for several random choices of n, d_{kq}, d_v.
3. **Agreement with PyTorch.** Construct the same computation with `torch.nn.functional.softmax` and matrix multiplies, and check your NumPy output matches to within 10^{-6}. Note: do *not* use `torch.nn.MultiheadAttention` for this — its scaling and projection conventions differ from ours and you will spend an hour reconciling them. Build the reference directly from the formula.
4. Numerical stability: entries of $\boldsymbol{XQ}$ and $\boldsymbol{XK}$ of magnitude $\sim 10^3$ should not produce `nan`.
5. The permutation property you derived in Q5.

Task 4: one observation

Pick $n = 6$, $d_{kq} = 2$, $d_v = 3$. Generate random $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ with entries from $\mathcal{N}(0, 1)$ and set $\boldsymbol{X} = I$. Print $\boldsymbol{W}$. Then scale $\boldsymbol{Q}$ and $\boldsymbol{K}$ by a factor c and print $\boldsymbol{W}$ again for $c \in \{0.1, 1, 10\}$. Describe what happens to $\boldsymbol{W}$ as c grows and as $c \to 0$.

As c grows the attention weights sharpen. For example at $c = 10$ each row is nearly one-hot (which means that one entry is close to 1 and the rest are close to 0), since scaling up the dot products before softmax pushes it toward the max entry. As c shrinks ($c = 0.1$), the rows flatten toward uniform ($\sim 1/6$ each), since small dot products leave softmax's input nearly constant across the row. At $c = 1$ the rows are somewhere in between (clearly non-uniform but not collapsed.)

This behaviour is worth understanding now; it comes back when we study how well $\boldsymbol{W}$ can be approximated by a low-rank matrix.

5 Write-up

Fill in the answer boxes above, then complete the sections below. Commit this `.tex` file to the repository along with your code, and send me the compiled PDF 24 hours before our meeting.

W1. What I did

Two or three sentences. Not a list of hours; a list of things that now exist.

I implemented and tested a from-scratch NumPy self-attention layer (softmax, attention_weights, self_attention) with a 5-test suite covering row-sum correctness, output shapes, agreement with PyTorch, numerical stability, and the permutation property. I also worked through the shape, role, and permutation-invariance questions on paper. Finally I have some better understanding of transformers from watching the 3 Simons' Institute YouTube videos, Karpathy YouTube video, and reading the Attention is all you need paper again for section 3.2.

W2. One claim, restated from memory

Close the video, close the paper, close your code. Write down the definition of a self-attention layer and explain in three or four sentences what it computes. Then reopen your sources and note anything you got wrong.

A self-attention layer takes a sequence of n vectors and, for each one, produces a new vector that is a weighted average of the values of every other vector in the sequence. The weights come from comparing that vector's query against every vector's key with a dot product, then normalizing with softmax so the weights are non-negative and sum to 1. After checking with my sources again I generally had this right except I initially forgot that the normalization is row-wise, not over the whole matrix, until I re-checked the formal softmax definition.

W3. Where I am stuck

Be specific. "I don't understand attention" is not usable. "I can't see why the $\boldsymbol{X}$ appears three times in the formula rather than once" is usable, and I can answer it in thirty seconds at the board.

If nothing is unclear, say so — but then write down a question you could not answer instead. There should always be something in this box.

I understand why we need separate Q and K matrices instead of just one. However I am still confused regarding why the value vectors need their own separate matrix V instead of just reusing the raw X (or the key vectors) as what gets aggregated. I still can't explain why decoupling "what I offer to be found" (K) from "what I actually hand over" (V) is necessary instead of than just convenient.

W4. Hours logged

Roughly how long on reading, coding, and writing.

Reading took about 5 hours total (1 hour refreshing on Attention Is All You Need section 3.2, since I'd already read the paper before our last meeting; 3 hours on the Simons Institute boot camp videos; 1 hour on the Karpathy self-attention segment). Coding took about 3-4 hours, working through softmax, attention_weights, and self_attention, using AI assistance but making sure I understood each step before moving on. Writing took the shortest amount of time at about 1 hour because this was mostly learning how LaTeX worked and reflecting on how the derivations worked right after finishing implementing the code.

W5. Plan for next week

One or two lines on what you expect to do next.

Next week I expect to continue rereading the Attention is all you need Vaswani et al. paper beyond Section 3.2 and start connecting self-attention to the full Transformer block (multi-head attention, positional encoding, and the feedforward layer). I realized that there were some key insights I missed after reading the paper the first time.

Meeting preparation

At the start of our meeting you will spend ten minutes at the board, with no notes, explaining self-attention. Assume I know linear algebra and nothing about transformers. Practice this out loud at least once beforehand; it is different from writing it down.

Checklist

☐ Repository created and shared

☐ `src/attention.py` implemented

☐ `tests/test_attention.py` passing, including the PyTorch agreement test

☐ `notebook.md` started, with a dated entry for this week

☐ Q1–Q5 and Task 4 answered in this file

☐ W1–W5 completed

☐ PDF sent 24 hours before the meeting

☐ Board explanation rehearsed